

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1510

COURSE OUTCOME COVERED

CO4: Analyze big data characteristics and challenges across domains including security and application aspects. (BL4)

Dave's Taxonomy: Precision (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

NAME: K. Saipriya

REGNO: 192421296

COURSE CODE: CSA1510

COURSE NAME: Cloud Computing and Big Data Analytics

Reverse Engineering Task

1. Big Data Platform for Website Clicks, Mobile Interactions and Transactions

Reverse-Engineered Architecture

The platform is likely designed as a distributed Lambda/Kappa-style big data architecture because it handles billions of events, requires large-scale storage, batch processing, and real-time analytics. The probable data flow is:

Website/Mobile/Transaction Sources → Event Ingestion → Distributed Storage → Batch & Stream Processing → Analytics Layer → BI Dashboards

1. Data Ingestion

The platform would probably use Apache Kafka as the main event-ingestion framework. Website clicks, mobile interactions, and transaction events can be published as Kafka topics.

For example:

- website_clicks
- mobile_events
- transactions
- customer_events

Kafka is suitable because it supports high-throughput event ingestion and allows multiple consumers to process the same events independently.

A distributed ingestion system such as Apache Flume or Kafka Connect could also be used to collect logs from web servers and applications.

2. Distributed Storage

The probable distributed storage layer is Hadoop Distributed File System (HDFS) or a cloud object store such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage.

Raw data can be stored in a data lake using formats such as:

- JSON for semi-structured clickstream data
- CSV for simple transaction exports
- Parquet/ORC for optimized analytical processing

HDFS or cloud object storage distributes data across multiple nodes, allowing the system to store billions of records without depending on a single server.

3. Separation of Structured and Semi-Structured Data

Structured transaction data would probably be stored in relational or analytical tables containing fields such as:

Customer_ID, Product_ID, Quantity, Price, Transaction_Date

Semi-structured clickstream data may contain:

Customer_ID, URL, Device, Timestamp, Browser, Location

A data lake can maintain separate zones:

Raw Zone → Cleaned Zone → Curated/Analytics Zone

Parquet can be used for structured analytical datasets because it is column-oriented and reduces the amount of data scanned during queries.

Data can also be partitioned by:

Date → Region → Event Type

For example:

/transactions/year=2026/month=08/day=18/

This improves query performance because analytics engines can skip irrelevant partitions.

4. Batch Processing

Apache Spark is a likely batch-processing framework.

Spark can perform:

- Customer behavior analysis
- Daily sales aggregation
- Clickstream analysis
- Customer segmentation
- Historical trend analysis

Hive may also provide SQL-based querying over large datasets.

5. Real-Time Stream Processing

For real-time analytics, the platform could use:

- Kafka Streams
- Apache Spark Structured Streaming
- Apache Flink

For example, every new transaction can be processed immediately to calculate:

- Current sales
- Active users

- Popular products
- Conversion rate
- Suspicious activity

Flink or Spark Structured Streaming would be particularly suitable for stateful event processing and window-based analytics.

6. Scalability and Partitioning

The platform probably uses horizontal scaling rather than relying on a single powerful server.

Kafka partitions allow multiple consumers to process events simultaneously. Similarly, HDFS divides large files into blocks and distributes them across DataNodes.

Spark divides workloads into parallel tasks and executes them across worker nodes.

Replication is also likely used so that failure of one machine does not result in permanent data loss.

7. Analytics and Machine Learning

The analytics layer may use Spark MLlib or Python-based machine learning frameworks for:

- Customer segmentation
- Demand prediction
- Churn prediction
- Recommendation
- Fraud detection

Aggregated datasets can then be exposed through SQL query engines such as Presto/Trino or Hive.

8. Dashboards and Business Intelligence

The processed analytical data would probably be connected to BI tools such as:

- Power BI
- Tableau
- Looker

The dashboard does not normally query billions of raw events directly. Instead, it queries aggregated analytical tables or data marts.

For example:

Kafka → Spark/Flink → Aggregated Sales Table → Power BI → Management Dashboard

This provides KPIs such as total sales, conversion rate, active customers, regional performance, and product trends.

Conclusion

Therefore, the probable architecture is a distributed data lake using Kafka for ingestion, HDFS/S3 for storage, Spark/Flink for processing, Trino/Hive for querying, and Power BI/Tableau for visualization. Partitioning,

replication, parallel processing, and horizontal scaling allow the system to handle billions of events while supporting both historical and real-time analytics.

2. E-Commerce Platform for Cart, Checkout, Product Views and Payments

Reverse-Engineered Architecture

This platform requires real-time personalization, so the architecture is likely event-driven:

Web/Mobile Events → Kafka → Stream Processing → Customer Profile/Cache → Recommendation Engine → Application

At the same time, historical data flows into the data lake for deeper analytics.

1. Event Streaming

Apache Kafka is a probable event-streaming system.

Separate topics could represent:

- product_views
- cart_events
- checkout_events
- payment_events
- customer_events

Each event would contain a timestamp and customer/session identifier.

For example:

Customer_ID: C102

Product_ID: P45

Event: Add_to_Cart

Timestamp: 10:32:15

Region: Chennai

Kafka provides durable event storage and allows multiple downstream systems to consume the same event.

2. Customer Profiling Database

The system probably maintains a customer profile store containing:

- Customer information
- Previous purchases
- Browsing history
- Preferred categories
- Current session
- Recommendation features

A distributed NoSQL database such as Cassandra, MongoDB, DynamoDB, or HBase could be used because customer profiles need fast and scalable access.

3. Real-Time Synchronization

Streaming engines such as Apache Flink or Spark Structured Streaming can consume Kafka events.

For example:

Product View → Kafka → Flink → Update Customer Profile → Recommendation Engine

When a customer views several mobile phones, the system can immediately update their behavioral profile.

If the customer then adds one phone to the cart, the recommendation system can use the updated profile to suggest accessories or similar products.

4. Caching

A system such as Redis is likely used as a low-latency caching layer.

Frequently accessed information could include:

- Product details
- Customer session
- Shopping cart
- Recommendations
- Popular products

Caching reduces database load and provides faster responses to customers.

5. Recommendation Workflow

The recommendation pipeline probably contains several stages:

Event Collection → Feature Engineering → Model Training → Recommendation Generation → Real-Time Serving

Historical data can be used to train recommendation models.

Possible approaches include:

- Collaborative filtering
- Content-based recommendation
- Association-rule mining
- Deep learning recommendation models

For example, purchase history can identify customers who bought product A and later purchased product B.

6. Distributed Query Engine

A distributed SQL engine such as Trino/Presto can query large datasets stored in the data lake without moving all the data to a traditional database.

It can combine data from:

- Transaction tables
- Clickstream logs
- Customer profiles
- Product catalog
- Marketing data

7. Structured and Semi-Structured Data Integration

Structured order information may look like:

Order_ID, Customer_ID, Product_ID, Amount, Date

Browsing events are more flexible and may be stored as JSON.

The analytics pipeline can transform both into a common analytical schema.

For example:

Browsing Data + Customer ID + Order Data → Customer 360 Dataset

This allows the company to determine whether product browsing actually leads to purchases.

8. Scalability and Fault Tolerance

Kafka partitions allow events to be processed in parallel.

NoSQL databases can distribute customer profiles across multiple nodes.

Redis can use replication and clustering.

Spark/Flink can recover failed processing tasks through checkpointing and fault-tolerance mechanisms.

Data can also be replicated across availability zones to reduce the effect of hardware failures.

9. KPI Monitoring

Processed data can be stored in an analytical warehouse or data mart.

BI tools can then generate dashboards showing:

- Conversion rate
- Cart abandonment rate
- Revenue
- Average order value
- Product popularity
- Regional sales
- Customer acquisition
- Payment success rate

For example:

Kafka → Flink → Aggregation → Data Warehouse → Power BI/Tableau → KPI Dashboard

Conclusion

The probable architecture combines Kafka, Flink/Spark, Redis, NoSQL databases, Trino/Presto, machine learning, and BI tools. The main design priority is low latency, because customer behavior must be converted into personalized recommendations almost immediately.

3. Big Data Healthcare System with Patient Histories, Images, Sensors and Prescriptions

Reverse-Engineered Architecture

Healthcare data is highly heterogeneous. Therefore, the system would most likely use a hybrid architecture combining relational databases, NoSQL storage, object storage, streaming systems, and analytics platforms.

The probable architecture is:

Hospitals/Labs/Wearables → Integration Layer → Data Lake + Clinical Database → Stream/Batch Processing → ML & Analytics → Clinical Applications

1. Hybrid Database Architecture

Structured information such as:

- Patient ID
- Diagnosis
- Prescription
- Appointment
- Laboratory result
- Billing information

can be stored in relational databases such as PostgreSQL or MySQL.

Unstructured information such as:

- CT scans
- MRI images
- X-rays
- Medical documents

would be better stored in object storage or a distributed file system.

A NoSQL database could store flexible clinical documents and sensor records.

Therefore, the system is likely to use:

Relational DB + NoSQL DB + Object Storage/Data Lake

2. Healthcare Data Integration

Hospitals, laboratories, pharmacies, and insurance systems use different data formats.

An integration layer would probably use healthcare interoperability standards such as:

- HL7
- FHIR
- DICOM

FHIR can exchange structured healthcare resources, while DICOM is widely used for medical imaging.

An integration platform or ETL system can extract, transform, validate, and load information from different healthcare organizations.

3. Data Ingestion

Wearable devices continuously generate:

- Heart rate
- Blood oxygen
- Temperature
- Activity
- Sleep data

A streaming platform such as Kafka can ingest these events.

For example:

Wearable → Gateway → Kafka → Flink → Alert System

If an abnormal heart-rate pattern is detected, the monitoring system can generate an alert.

4. Machine Learning

The ML pipeline could include:

Data Collection → Cleaning → Feature Engineering → Model Training → Validation → Deployment → Monitoring

Models may be used for:

- Disease risk prediction
- Readmission prediction
- Patient deterioration detection
- Medical image classification
- Treatment outcome prediction

For medical images, deep learning models such as CNN-based architectures could be used.

5. Distributed Processing

Apache Spark can process millions of patient records for population-level studies.

For example, researchers can analyze:

- Disease prevalence
- Regional disease patterns
- Treatment outcomes
- Risk factors
- Hospital readmission rates

Instead of processing all records on one machine, Spark distributes the computation across multiple worker nodes.

6. Privacy and Encryption

Healthcare data is highly sensitive, so security must be integrated throughout the architecture.

Probable controls include:

- Encryption at rest
- Encryption in transit using TLS
- Role-Based Access Control
- Multi-factor authentication
- Data masking
- Tokenization
- Audit logging

For example, doctors may access clinical information required for treatment, while analysts may receive de-identified datasets.

7. Compliance

The platform would need to comply with applicable healthcare data-protection regulations.

Depending on the country and deployment environment, this may include requirements related to:

- Patient consent
- Data minimization
- Access control
- Audit trails
- Retention
- Secure data sharing

8. Clinical Analytics

The analytics pipeline can combine:

Patient History + Lab Results + Wearable Data + Prescriptions + Imaging

to generate a patient-level clinical view.

For example:

Wearable abnormality → Stream Processing → Risk Model → High-risk Alert → Clinician Dashboard

Population-level analytics can separately use historical data for research.

Conclusion

The most probable solution is a hybrid healthcare data platform using relational databases for structured records, object storage for medical images, NoSQL for flexible data, Kafka/Flink for sensor streams, Spark for large-scale analytics, and ML frameworks for prediction. Strong encryption, authentication, access control, auditing, and healthcare interoperability standards are essential because the system handles sensitive patient information.

4. Smart City Platform for Traffic, CCTV, Weather and Public Transport

Reverse-Engineered Architecture

A smart city system requires extremely low-latency processing because traffic conditions can change within seconds. Therefore, the probable architecture uses edge computing + distributed streaming + centralized data lake + real-time analytics.

Sensors/CCTV/Transport/Weather → Edge Layer → Kafka → Stream Processing → Real-Time Storage → Dashboard

Historical data simultaneously flows into batch analytics.

1. Edge Computing

Traffic cameras and sensors generate huge amounts of information.

Sending every raw video frame to a central cloud would consume significant bandwidth. Therefore, edge devices probably perform initial processing.

Edge processing can extract:

- Vehicle count
- Vehicle speed
- Traffic density
- Accident indicators
- License plate metadata where legally permitted

Only relevant metadata may then be transmitted to the central platform.

2. Distributed Messaging

Apache Kafka is a probable messaging platform.

Topics may include:

- traffic_sensor
- weather
- public_transport
- cctv_metadata

Kafka partitions allow thousands of sensors to send data concurrently.

3. Centralized Storage

Historical information could be stored in:

- HDFS
- Amazon S3
- Azure Data Lake
- Google Cloud Storage

The data lake could contain several years of traffic and environmental information.

4. Time-Series Database

Traffic sensors generate timestamp-based data, so a time-series database such as:

- InfluxDB
- TimescaleDB

could be used.

For example:

Sensor_ID | Timestamp | Vehicle_Count | Average_Speed

This allows efficient time-range queries.

5. Geospatial Database

Traffic data is also location-dependent.

A spatially enabled database such as **PostGIS** can support queries such as:

- Traffic within a particular area
- Nearest traffic sensor
- Congestion around a road
- Vehicles entering a geographic region

This makes geospatial analytics possible.

6. Real-Time Stream Processing

Flink or Spark Structured Streaming could calculate:

- Current traffic density
- Average speed
- Congestion level
- Abnormal traffic patterns

For example:

Traffic Sensors → Kafka → Flink → Congestion Detection → Control Center

Window-based processing can calculate average traffic over the last 5 or 10 minutes.

7. Congestion Prediction

Machine learning can combine:

Historical Traffic + Weather + Time + Day + Events + Public Transport

to predict future congestion.

Models may include:

- Random Forest
- Gradient Boosting
- Time-series models
- Neural networks

8. Batch and Real-Time Analytics

Real-time processing is useful for immediate operational decisions.

Batch processing is useful for long-term planning.

For example:

Real-Time: Detect a traffic jam and alert traffic operators.

Batch: Analyze six months of traffic patterns to decide where a new road or traffic signal is required.

Thus, both processing methods complement each other.

9. Security and Access Control

Smart city platforms contain sensitive infrastructure and potentially sensitive citizen-related information.

Probable controls include:

- Encryption
- API authentication
- Role-Based Access Control

- Network segmentation
- Device authentication
- Audit logs
- Secure communication between sensors and servers

Different users should have different privileges. For example, traffic operators may access live traffic information while city planners may only access aggregated historical statistics.

10. Visualization

Operational dashboards may use:

- Grafana
- Power BI
- Tableau

A city control center could display:

- Live traffic maps
- Congestion alerts
- Bus locations
- Weather conditions
- Accident alerts
- Sensor health

A geospatial map can use different indicators to show congestion areas.

Conclusion

The probable smart-city architecture combines edge computing, Kafka, Flink/Spark, time-series databases, PostGIS, cloud/data-lake storage, machine learning, and visualization platforms. Edge processing reduces network traffic, while distributed streaming provides low-latency analysis and centralized storage supports long-term urban planning.

5. Healthcare Analytics System for Patient Records, Wearables, Labs and Appointments

Reverse-Engineered Architecture

This system is similar to Question 3 but focuses more strongly on health analytics, interoperability, population-level processing, governance, and ML-based risk scoring.

The probable architecture is:

Hospitals/Clinics/Wearables/Labs → ETL & Interoperability Layer → Hybrid Storage → Real-Time + Batch Processing → ML → Clinical Dashboard

1. Hybrid Storage Architecture

Patient records and appointment information are structured and can be stored in a relational database.

For example:

Patient_ID

Name

Age

Diagnosis

Appointment_Date

Doctor_ID

Lab reports may contain semi-structured information, while medical documents and images are unstructured.

Therefore, a combination of:

Relational Database + NoSQL + Data Lake/Object Storage

is likely.

This is preferable to forcing every type of medical information into a single database.

2. ETL Pipeline

The ETL process probably consists of:

Extract → Validate → Transform → Standardize → Load

Data can arrive from:

- Hospital systems
- Clinics
- Laboratories
- Wearable devices
- Appointment systems

The integration layer standardizes identifiers, timestamps, medical codes, and patient information before storing it.

3. Interoperability

Healthcare interoperability standards such as **HL7** and **FHIR** are probable because different hospitals and clinical applications need a common way to exchange patient information.

Medical imaging systems may use **DICOM**.

This reduces the problem of incompatible healthcare systems.

4. Real-Time Monitoring

Wearables continuously generate patient readings.

The probable real-time pipeline is:

Wearable → Kafka → Flink/Spark Streaming → Feature Extraction → ML Model → Alert

For example, a persistent abnormal heart-rate pattern could increase a patient's risk score and notify authorized healthcare personnel.

5. Distributed Processing

For population-level analysis, **Apache Spark** is a likely framework.

Spark can process large datasets containing millions of:

- Patient records
- Lab results
- Diagnoses
- Prescriptions
- Wearable readings

It can identify trends such as increasing disease rates or relationships between patient characteristics and treatment outcomes.

6. Machine Learning Pipeline

The ML pipeline would likely contain:

Data Preparation → Feature Engineering → Model Training → Validation → Deployment → Monitoring

Possible models include:

- Logistic Regression
- Random Forest
- Gradient Boosting
- Neural Networks

For example, patient age, previous diagnosis, lab values, medication history, and wearable measurements can be converted into features for a disease-risk model.

7. Diagnosis Risk Scoring

The system could generate a probability/risk score rather than directly replacing a clinician's decision.

For example:

Patient Data

↓

Feature Extraction

↓

ML Risk Model

↓

Risk Score

↓

Clinical Review

A high-risk score can prioritize a patient for further clinical assessment.

8. Treatment Recommendation

A recommendation system may compare:

- Patient history
- Diagnosis
- Lab results
- Previous treatments
- Medication response

to provide decision-support information.

However, the final treatment decision should remain with qualified healthcare professionals.

9. Encryption and Compliance

Because medical information is sensitive, encryption is likely applied at two levels:

Data in Transit → TLS/HTTPS

Data at Rest → Database/Object Storage Encryption

Additional controls include:

- Role-Based Access Control
- Multi-factor authentication
- Audit logs
- Data masking
- De-identification
- Key management
- Patient consent management

Every access to sensitive patient information can be recorded for auditing.

10. Data Governance

A governance layer can define:

- Who can access data
- What data can be accessed
- How long data is retained
- Whether data can be shared
- How patient identity is protected
- How data quality is monitored

De-identified datasets can be provided to researchers instead of exposing direct patient identifiers.

11. Analytics Dashboard

The final analytics layer can provide dashboards for different users.

Doctor Dashboard:

- Patient risk score
- Recent lab results
- Wearable alerts
- Appointment history

Hospital Dashboard:

- Patient admissions
- Disease trends
- Resource utilization

Population Health Dashboard:

- Disease prevalence
- Regional trends
- Treatment outcomes

Conclusion

The most probable architecture is a hybrid healthcare analytics platform combining relational databases, NoSQL storage, data lakes, Kafka, Spark/Flink, FHIR/HL7 interoperability, machine learning, and BI dashboards. Distributed processing supports population-scale analytics, while encryption, access control, auditing, and de-identification protect sensitive medical information.

